ReconciCA: Auditable GST Purchase Reconciliation with Explicit Ambiguity

1st Murtaza Zaki
Dept. of Computer Science & Eng.
G H Raisoni College of Eng.
Nagpur, India
murtaza.zaki.cse@ghrce.raisoni.net

2nd Piyush Katankar
Dept. of Computer Science & Eng.
G H Raisoni College of Eng.
Nagpur, India
piyush.katankar.cse@ghrce.raisoni.net

3rd Aman Khairkar
Dept. of Computer Science & Eng.
G H Raisoni College of Eng.
Nagpur, India
aman.khairkar.cse@ghrce.raisoni.net

4th Dhananjay Bhoyar
Dept. of Computer Science & Eng.
G H Raisoni College of Eng.
Nagpur, India
dhananjay.bhoyar.cse@ghrce.raisoni.net

5th Dr. Girish Talmale
Assistant Professor, Dept. of Computer Science & Eng.
G H Raisoni College of Eng.
Nagpur, India
girish.talmale@raisoni.net

***Abstract*— Before a business claims Input Tax Credit (ITC) it must compare the GSTR-2B form with its Purchase Register. The numbers on the invoices the dates and the tax amounts often do not match exactly. When a spreadsheet looks up the matching record it can pair the wrong invoices together. This paper introduces ReconciCA, a browser-based application that uses a Dart matching engine. The engine groups records by the supplier's GSTIN then checks the invoice number, the date and the five tax components. It uses each record once and flags any ties that need human review. An optional component called the Purchase Compiler reads bills using a vision-language model. The results from that model must be. Reviewed by a person before they can be exported. The account layer of ReconciCA stores private bill copies keeps a purchase ledger. Maintains a cache for replies, from the model. The cache only reuses a reply when the exact bill bytes and the request settings match; the hit rate and cost savings of this cache have not been measured. On a test set of 50 invoice-number pairs that the authors labeled the held-out results at a threshold of 0.80 were 0.765 precision, 1.000 recall and 0.867 F1. Those numbers apply to this tiny set; they do not prove that the system works well in production. When we shuffled the input the decisions stayed the same for the 26 paired reconciliation cases. Still a search that always takes the record first can produce a false match as illustrated by one example. All reconciliation remains inside the browser. When the optional bill-reading feature is used the selected files are sent to services.**

***Index Terms*—GST reconciliation, GSTR-2B, input tax credit, record linkage, ambiguity, auditability, invoice extraction**

I. Introduction

India's Goods and Services Tax law has rules for claiming Input Tax Credit and these rules depend on the details provided in supplier invoices [1]. GSTR-2B is the portal's automatically generated ITC statement [2]. Businesses need to verify that every entry in GSTR-2B is also present, in their Purchase Register. This helps ensure accuracy and compliance.

This is a record linkage task. A wrong match can hide a missing invoice. Invoice numbers may contain typos dates may. Rounding can change amounts. A first-result lookup can make a match without warning. ReconciCA aims to return the result for the same inputs compare money as integers, link results to source rows and flag cases that need review. The reconciliation engine runs in the browser. Users without a register can also use optional Purchase Compiler. It creates draft rows from bills. Signed-in users can retain bills and saved purchase entries across sessions.

A. Motivation

The system must return the result regardless of how the files are arranged. The system must compare amounts reliably explain every outcome, use each invoice once and keep reconciliation files on the user's device. When two candidates look equally likely the system should flag the tie of guessing..

B. Key Contributions

This paper makes four contributions. First, it describes a set of matching stages that groups records by GSTIN and checks same-date candidates first. Second, it assigns each record at most once and reports ties. Third, it presents a browser app that links each result to its source rows. Fourth, it measures selected behaviors and shows a case where the stage order leads to a wrong match. The optional compiler is part of the app, but its invoice-reading accuracy has not been measured.

II. Related Work

A. Record Linkage

Record linkage tries to figure out if entries from sources are about the same real-world thing. Recent studies look at how blocking keys can group records to reduce the number of comparisons needed between pairs. They also explore ways to pick the blocking keys [3]. A recent survey goes over methods, for matching entities when data comes from sources and has different formats [4]. ReconciCA uses the supplier GSTIN as its blocking key and relies on clear fixed rules. It does not learn patterns from data. Ensure that the final matches are the best possible assignments..

B. Comparing Strings

Levenshtein distance counts insertions, deletions, and substitutions between strings [5]. Jaro and Jaro–Winkler are well known comparison methods. [6], [7].We compare them using our small labelled set. This paper does not propose a new string measure.

TABLE I
RECONCILIATION OUTCOMES

Outcome	Interpretation
GREEN	Paired according to the configured rules
AMBER	Needs review because of a discrepancy or ambiguity
RED	Purchase Register document has no GSTR-2B match
PURPLE	GSTR-2B document has no Purchase Register match

C. GST Reconciliation Tools

Businesses already have to compare purchase records with data. This work looks at how the app handles ties. It also looks at whether the file order changes its decisions. The study checks how the app verifies each tax component. It also examines how a user can trace a result back, to its source.

D. Reading Invoices with Large Models

Recent work tests methods for handling invoices both those that use images and text and those that only use text across many datasets [8]. This research gives some background for the compiler but it doesn't test the GLM-4.6V pipeline that this application uses. The Purchase Compiler uses GLM-4.6V to read bills; the results, from GLM-4.6V are not. Must go through the apps checks before being exported [9].

E. Research Gap

This paper evaluates the process handling and input-order behavior in this implementation. It does not claim that these topics have not been studied in other GST reconciliation systems.

III. PROBLEM FORMULATION

Let G be the GSTR-2B invoices and P the Purchase Register rows. Each record has a supplier GSTIN, invoice number, date, taxable value, and IGST, CGST, SGST, and cess amounts. Purchase rows with the same normalized GSTIN and invoice number are combined. The engine gives each document one outcome: Each document can appear in only one match. The system also keeps the source-row IDs. This study covers B2B invoices from one filing period. It does not cover amendments, credit or debit notes, ISD credit, or imports.

IV. THE RECONCILIATION ENGINE

Fig. 1 shows how data moves through the app. The Flutter web client reads and cleans the input, runs the matching process, and shows the results. The core Dart engine has no network or file-system access. Only the optional compiler sends a selected bill to an outside model service; signed-in use also stores bill and ledger data in the account service.

A. Cleaning, Folding and Money

The app trims Invoice numbers. It changes all letters to case. It also reduces spaces. Punctuation and leading zeroes stay unchanged. The app adds tax-rate lines. Purchase rows for the same invoice. It keeps each tax component and its source details intact. If there are repeated purchase rows, with dates it uses the earliest date and records a warning. Amounts are converted from rupees to integer paise before comparison. This avoids floating-point math during tolerance checks.. Parsing a decimal amount still involves rounding.

B. GSTIN Check Character

If a GSTIN fails its check-character test, the app reports a data-quality issue. It does not reject a possible match for this reason. The GSTIN still groups the supplier's records.

C. Blocking and the Two-Tier Date Search

I see that the engine groups records by the GSTIN so the engine does not compare invoices that come from different suppliers. This is the engine's blocking key; blocking keys group records to reduce the number of pairwise comparisons [3]. The engine checks candidates that share the date first and then it looks at the remaining records that are within 30 days of each other. This order can fail. A fuzzy match, on the date can use a record that could have matched an exact invoice number on another date. The evaluation shows one case. Consistent results do not guarantee that the match is correct or the best match.

D. Comparing Invoice Numbers

For invoice numbers a and b, the similarity score is $s(a, b) = 1 - d(a, b)/\max(|a|, |b|)$. Here, d is the Levenshtein distance, found with a two-row dynamic program [5]. Identical strings are handled directly. The fuzzy-match threshold is 0.80. A close number is not enough to accept a match; the date and every tax component must also pass.

E. Comparing Amounts

Let g and p be the matching tax amounts from GSTR-2B and the Purchase Register, in paisa. The exact rule allows $|g - p| \leq 1$. The relative rule allows $10{,}000|g-p| \leq 50|g|$, or 0.5%. Every component must pass its rule. These integer checks also avoid division at the threshold.

F. The Decision Cascade

The engine uses the rule that matches. Stage 1 looks for an invoice number on the same date when each tax amount is within one paisa. Stage 2 looks for a number on the same date when all amounts are within the relative limit. This match is labeled as confidence. A possible match that has the day amount conflict or a date difference of up to 30 days is marked AMBER for review. Rows in the Purchase Register that do not match are RED. Rows, PURPLE in GSTR-2B that don'tmatch

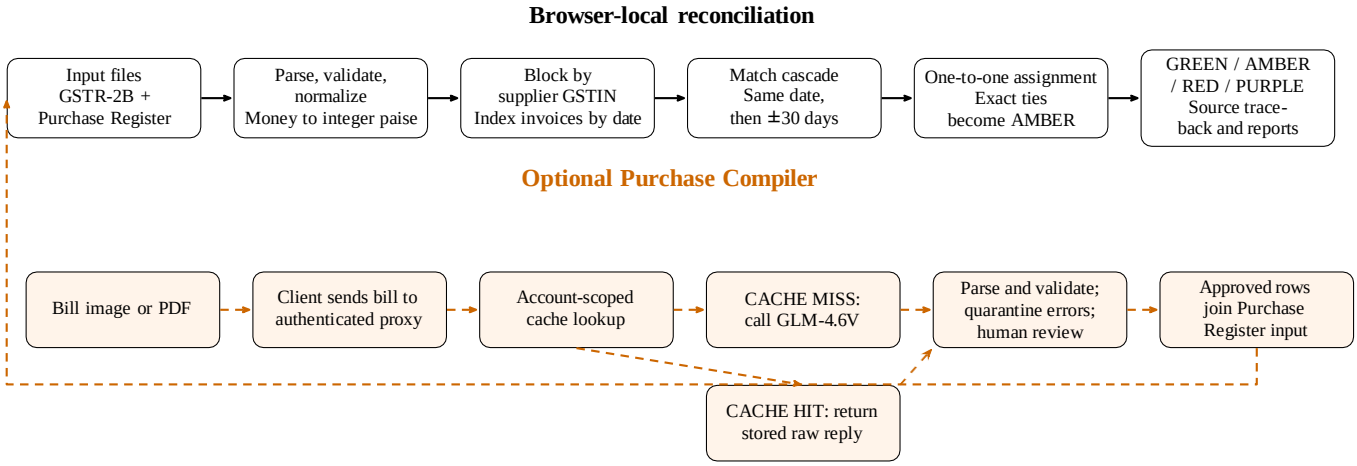


Fig. 1. ReconciCA processing flow. Bill reading uses an optional external service and an account-scoped cache; a cache hit returns the stored reply for validation, while a miss invokes the model. Regular reconciliation runs in browser memory.

TABLE II
CANDIDATE RULES WITHIN THE SAME GSTIN

Stage	Candidate condition	Status
1	Same date and exact number; each component differs by at most 1 paisa	GREEN
2	Same date; $s \geq 0.80$; each component within 0.5%	GREEN*
3b	Same date and plausible number, but a component conflicts	AMBER
3	Plausible number with a date gap of at most 30 days	AMBER
4/5	No counterpart is assigned	RED / PURPLE

Status key: GREEN = matched; GREEN* = low-confidence fuzzy match; AMBER = review or ambiguity; RED = unmatched Purchase Register item; PURPLE = unmatched GSTR-2B item.

G. One-to-One Assignment with Explicit Ambiguity

The engine sorts candidates based on how similar they're then based on how much taxes differ, then based on how far apart the dates are. It takes a pair only if neither of the two records has been used before. If candidates have the rank it puts the linked pairs into groups. A group that has one pair is approved. A group that has pairs that compete with each other is marked as AMBER for checking. This deals with matches, without depending on the order in which things are processed. However the way it works does not make sure that the possible assignment is made.

H. A Failure of the Tier Order

The stage order has a known problem. A pair that has the date but similar but different invoice numbers can be accepted before the system checks for an exact match, on another date. This leads to a green status and leaves the right invoice without a match. So stable results do not always mean results. A future version could place importance on the exact cross-date match or mark the conflict for review.

V. THE PURCHASE COMPILER

The optional Purchase Compiler turns bill images or PDFs into draft Purchase Register rows. It checks the file type from its contents, then sends the document through an authenticated proxy to GLM-4.6V. The app looks at the response. Makes sure all the important parts are there. It checks things like the required fields, dates, GSTINs, amounts and the total that is, on the bill. People can make changes to the draft before they export it. If one invoice does not pass the checks the app keeps it. Shows why; the other invoices can still be looked at. The models output is a suggestion. When someone is signed in the app saves the bills and the purchase entries that are accepted into the account service. The step where the app reads the bill sends data outside the browser. The accuracy of this step has not been tested on a group of real bills that is representative.

A. Checking the File

Before the upload, app checks the file contents to see whether they are a supported image or PDF. It rejects unsupported files, even when the filename has an accepted extension.

B. Reading the Bill

The request sends the document data with a list of fields to extract. The response reader handles common wrapper text but rejects unusable results. The proxy keeps the model-service key. A bill leaves the device only when the user chooses this feature.

C. Account Persistence and Extraction Cache

The account layer stores original bills in a private bucket and accepted purchase entries in an account-scoped ledger. Its read-through cache made by the account, SHA-256 of the original bill bytes, fingerprints of the prompt, response schema and bill-preparation settings, and the model identifier. The cache stores the model's raw response. When there is a cache hit, the app re-parses that response again. Any change to a request component results in a cache key. A changed request component builds a different key. Once replies completed, structurally valid replies which include extracted invoices are cached. Simultaneous identical misses can still make duplicate model callsThe cache prevents making the inference call twice for a matching valid entry.. The implementation does not measure hit rate or show any cost-saving results. This paper focuses on the cache design and how the code behaves during production.

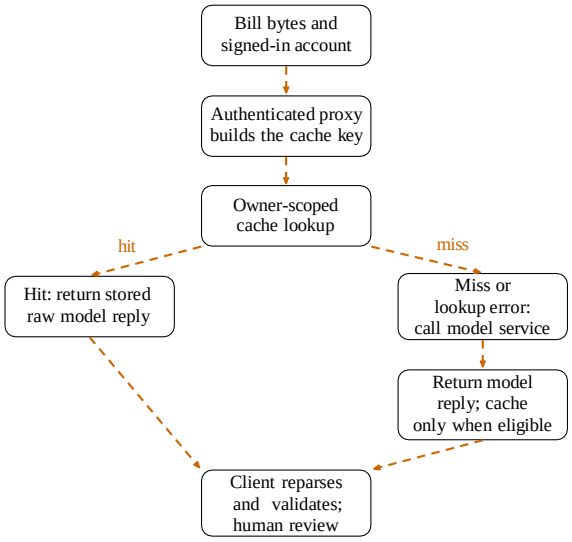


Fig. 2. Implemented extraction-cache path. A hit returns the stored raw reply; a miss or lookup error calls the model. The proxy caches a valid, non-empty reply when eligible, while returning the model response either way. Both paths feed client-side parsing, validation, and human review. This diagram reports system flow, not cache performance.

TABLE III
HELD-OUT INVOICE-NUMBER RESULT (N = 20)

Measure	Estimate	95% interval
Precision	0.765	[0.527, 0.904]
Recall	1.000	[0.772, 1.000]
F1	0.867	[0.714, 0.971]

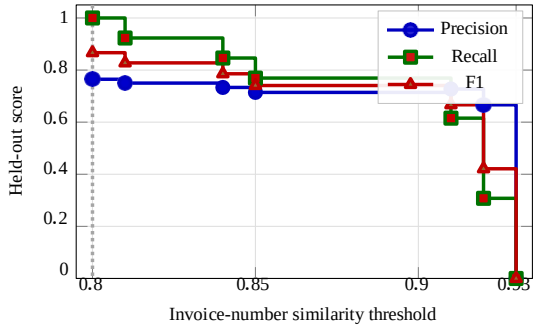


Fig. 3. Held-out precision, recall, and F1 at thresholds that change a prediction (0.80–0.93). The dotted line marks the threshold selected using the tuning set (0.80).

D. The Admission Check

An invoice can only be exported when the identity fields of the invoice the date of the invoice the supplier GSTIN of the invoice and the tax amounts of the invoice are present and pass the checks. These checks catch some mismatched values of the invoice. They cannot prove that an extracted value of the invoice is right so a person must still review the invoice., so a person must still review it.

E. Keeping Human Corrections Safe

The app keeps track of which fields came from the model and which were changed by the reviewer. Revision IDs make sure that a late response doesn't overwrite recent edits. Content IDs help detect if a file was uploaded more, than once during the session. These steps support the review process. They don't show how well the model understands real invoices.

VI. SUPPLIER-SCOPED REPORTS

The app can create a PDF for each supplier. It includes the supplier's RED and AMBER items. Does not include PURPLE items, from the buyer's own Purchase Register. Decisions are grouped by GSTIN before the PDF is generated. This paper focuses on the matching engine not on how easy users find the reports to use.

VII. EVALUATION

A. Data and Method

No real GSTR-2B export was available for the evaluation. The project corpus includes 45 cases: 26 paired reconciliation cases, 18 parse- cases and one manual-only case, with no generated files. The paired cases examine engine outcomes and input-order behavior; parse- cases test file handling. We also used 50 invoice-number pairs labelled by the authors:

33 were in the same invoice and 17 were of different invoices. These pairs were split once into 30 tuning examples. 20 Held-out examples. The threshold that gave the F1 score on the tuning set was chosen from values that were spaced by 0.01; the held-out set was not used to pick this threshold. The invoice-number pairs were manually. Not checked by a second reviewer so the results only apply to this small set. We used Wilson intervals to calculate confidence for proportions [10] and percentile bootstrap intervals, for F1 scores [11].

B. Matching Results

At threshold 0.80, the held-out set had 13 true positives, 4 false positives, no false negatives, and 3 true negatives. The scores were 0.765 precision, 1.000 recall, and 0.867 F1. The 95% bootstrap interval for F1 was 0.714–0.971. The point estimate did not reach the 0.90 target. The false positives show that a similar invoice number alone cannot confirm a match.

C. What the Labels Actually Say

At threshold 0.80 the held-out set had 13 positives, 4 false positives, no false negatives and 3 true negatives. The scores were 0.765 precision, 1.000 recall and 0.867 F1. The 95% bootstrap interval for F1 was 0.714, to 0.971. The point estimate did not reach the 0.90 target. The false positives show that a similar invoice number alone cannot confirm a match.

D. Comparing Similarity Measures

We looked at the 50 pairs and found that Jaro had a higher AUC than the edit-distance ratio that was implemented. The AUC for Jaro was 0.921 while the AUC for the edit-distance ratio was 0.701. Jaro-Winkler also had a held-out F1 than the current rule. The F1 for Jaro-Winkler was 0.897 whereas the F1 for the rule was 0.867.. This small author-labelled set is not enough to justify changing the production rule. We need a set, with independent labels..

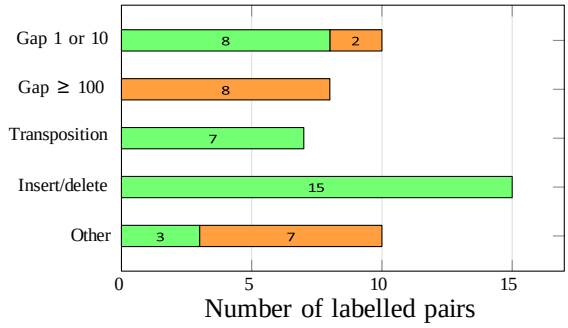


Fig. 4. Pairs labelled by the authors, grouped by edit pattern (green: same invoice; orange: different invoice). Small groups are combined. These labels apply only to this corpus.

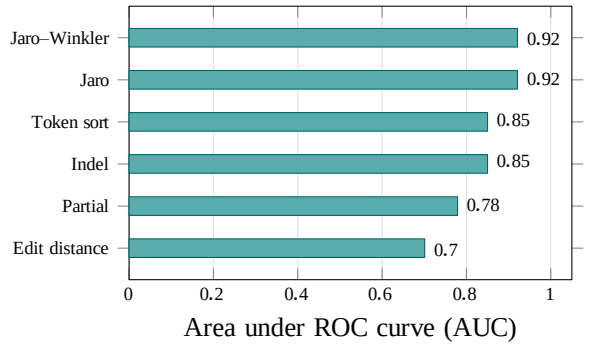


Fig. 5. AUC for six string measures on the 50 pairs labelled by the authors. Higher values mean better ranking in this set. The small sample limits wider claims.

E. Independence from Input Order

The input-order check used the corpus's 26 paired reconciliation cases. These 26 paired reconciliation cases contain 254 decisions in total. We shuffled supplier blocks, invoices and purchase rows 25 times for each case giving a total of 650 shuffles. The full engine returned the decisions every time. Without tie handling the result changed in 11 out of 25 shuffles for the one case that had a tie. This check covers these cases, not real accounting data, at scale.

F. Report Isolation

Tests check how suppliers are grouped if PDFs are created and if report pages are displayed properly. Other tests intentionally add data from suppliers or PURPLE rows and make sure the report identifies them. These tests look at how the software works; they do not look at how users feel when using it..

G. Speed

We ran the two-stage engine on 2,200 documents, per side one GSTIN and 28 dates. Over five Dart VM runs on an Apple M4 Pro the median time came out to 608 ms. An earlier flat-search version took around 7.7 seconds. It wasn't retested under the same conditions. This is a rough comparison, not a controlled benchmark.

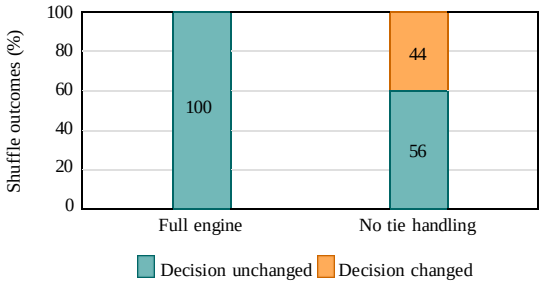


Fig. 6. Input-order check on the 26 paired reconciliation cases. The full engine kept the same decisions in all 650 shuffles. Without tie handling, the result stayed the same in 14 of 25 shuffles and changed in 11 for the one tied case.

H. Requirements and Tests

Tests check money-tolerance limits, one-to-one matching, ties, parsing and saving and reopening archives. Automated tests run across the engine, app and proxy. Passing these tests means the rules behave as expected.. It doesn't prove accuracy, on real accounting records. Reconciliation happens locally. The optional compiler sends selected documents to a service.

VIII. Conclusion

ReconciCA groups records by GSTIN checks each tax component assigns matches one to one and flags tied candidates. In the 26 paired cases changing input order did not change the engines decisions. The tests also found positives from similar invoice numbers and one wrong match caused by the same-date-first order. The small synthetic dataset cannot show how accurate the app will be in production. That needs a study with real, authorized records and independent labels. The matching stages also need to handle conflicts between a date fuzzy match and a stronger exact match, on another date. The optional Purchase Compiler still needs review. Its accuracy has not been measured..

References

[1] Government of India, *Central Goods and Services Tax Act, 2017*, sec. 16(2)(aa), inserted by the Finance Act, 2021. [Online]. Available: https://taxinformation.cbic.gov.in/content/html/tax_repository/gst/acts/2017_CGST_act/active/chapter5/section16_v1.00.html

[2] Goods and Services Tax Network, "FAQs: Viewing Form GSTR-2B," GST Portal User Guide. [Online]. Available: https://tutorial.gst.gov.in/userguide/returns/FAQ_gstr2b.htm

[3] D. C. do Nascimento, C. E. S. Pires, and T. P. da No´brega, "A generalized approach to perform unsupervised blocking key selection for entity resolution," *Information Sciences*, vol. 733, Art. no. 122952, Apr. 2026, doi: 10.1016/j.ins.2025.122952.

[4] M. H. Moslemi, A. Mousavi, B. Behkamal, and M. Milani, "Heterogeneity in entity matching: A survey and experimental analysis," *Data & Knowledge Engineering*, vol. 164, Art. no. 102575, Jul. 2026, doi: 10.1016/j.datak.2026.102575.

[5] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[6] M. A. Jaro, "Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida," *J. Amer. Statist. Assoc.*, vol. 84, no. 406, pp. 414–420, 1989, doi: 10.1080/01621459.1989.10478785.

[7] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi–Sunter model of record linkage,” in *Proc. Section Survey Res. Methods, Amer. Statist. Assoc.*, 1990, pp. 354–359.
[8] D. Berghaus, A. Berger, L. Hillebrand, K. Cvejoski, and R. Sifa, “Multi-modal vision vs. text-based parsing: Benchmarking LLM strategies for invoice processing,” arXiv:2509.04469, 2025. [Online]. Available: https://arxiv.org/abs/2509.04469
[9] Z.ai, “GLM-4.6V: Open Source Multimodal Models with Native Tool Use,” Dec. 8, 2025. [Online]. Available: https://z.ai/blog/glm-4.6v
[10] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.
[11] B. Efron, “Bootstrap methods: Another look at the jackknife,” *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.